

OpenGov

Salesforce Developer Assessment

Solution Design Document

Architecture · Components · Validations · Recommendations

1. Executive Summary

This solution automates the B2B Quote-to-Asset lifecycle on Salesforce CPQ. When a Sales Rep contracts a Quote, downstream automation creates Assets for every Hardware line, sets up Tasks for service onboarding, and surfaces all of it in a Customer Community where the customer can register warranties and track service requests.

The architecture prioritizes enterprise readiness: layered Apex (Trigger → Handler → Service), 100% bulkification proven with hard test assertions, layered validation (declarative + CPQ + Apex addError), and one error-cleaning utility reused across three UI surfaces (Flow Chatter, Apex DML, LWC modal).

2. Solution Components

Component	Type	Responsibility
Warranty_Registration__c	Custom Object	Tracks warranty term per Asset. Master-Detail to Account.
Quote_Contracted_Create_Assets	Record-Triggered Flow	Creates Assets from Hardware Quote Lines when Quote is Contracted.
AssetService.cls	Apex Service	Bulkified Task creation when Assets transition to Installed.
AssetTrigger / Handler	Apex	3-line dispatcher; delegates to service.
WarrantyRegistrationTrigger / Handler	Apex	Complex cross-object & cross-record validation via addError().
FlowErrorHandler.cls	Invocable Apex	Cleans raw fault messages; reused by Flow and LWC.
CommunityPortalController.cls	Apex (Aura-enabled)	Account-scoped read/write endpoints for community LWCs.
4 Lightning Web Components	LWC	Customer-facing Dashboard, Assets, Warranties, Service Requests.
Permission Set "Community Warranty Access"	Permission Set	Grants community user access to objects, fields, and Apex classes.

3. Architecture Pattern

3.1 Layered Apex

Every object follows a strict three-layer pattern:

- Trigger (3 lines): only dispatches to the handler. No logic.
- Handler: extends TriggerHandler base class. Routes context (beforeInsert, afterUpdate, etc.) to typed methods. No DML or queries.
- Service: all business logic, SOQL, DML. Reusable from any caller, not just triggers.

This separation means the Service class can be invoked from a Flow (via InvocableMethod), a REST endpoint, a Batch job, or a Queueable, without duplicating logic. The Trigger is just one of many possible entry points.

3.2 Cross-Surface Code Reuse

FlowErrorHandler is the keystone of the solution's reuse story. The same Apex class is called from three execution contexts:

- Record-Triggered Flow fault path → cleans the message before posting to Chatter.
- CommunityPortalController catch block → cleans DmlExceptions before re-throwing as AuraHandledException to the LWC.
- Available for any future Apex code that catches platform exceptions.

Three callers, one source of truth. A regex tweak in one place updates the customer experience everywhere.

3.3 Why Trigger Framework over Triggerless

A Trigger Handler framework was chosen over a pure-Flow approach for three reasons. First, bulkification: Apex handles 200+ records in a transaction without approaching CPU limits; Flows hit limits sooner. Second, recursion control: the framework includes a static bypass mechanism for data loads. Third, debuggability: Apex stack traces and unit tests give faster diagnosis than nested Flow fault paths.

4. Validation Strategy

Validation is layered. Simple field checks are declarative; complex cross-record/cross-object checks require Apex.

Layer	What It Catches	Why That Layer
Validation Rule	End Date ≤ Start Date on Warranty_Registration__c.	Simple field comparison — declarative is fastest and admin-maintainable.
CPQ Product Rule	Hardware Quote Lines without SBQQ__Description__c.	Fires inside the CPQ save lifecycle; gives a CPQ-styled error.
Apex addError (Cross-Object)	Warranty Asset belongs to a different Account than the Warranty.	Requires comparing related Asset.AccountId to current record — not viable in a formula.
Apex addError (Cross-Record)	Duplicate Active warranty for the same Asset.	Requires SOQL to query sibling records — impossible in any declarative tool.

5. Bulkification

`AssetService.createKickoffTasks` is the canonical example:

1. Collect unique AccountIds into a `Set<Id>` in a single pass through `Trigger.new`.
2. One SOQL query loads the Account → OwnerId map.
3. Build all Tasks in memory using the map for O(1) owner lookup.
4. Single DML inserts all tasks at once.
5. `Database.SaveResult` inspection logs per-row failures without rolling back the batch.

Proof via Unit Test

`AssetServiceTest.bulkInsert200AssetsHitsGovernorLimitsCleanly` inserts 200 Assets in a single transaction and asserts:

```
System.assert(Limits.getQueries() < 10);  
System.assert(Limits.getDmlStatements() < 10);
```

Hard numeric assertions catch regression: if a future developer adds a SOQL inside a loop, this test fails before the code ships.

6. Record-Triggered Flow: Quote → Asset

Trigger object: SBQQ__Quote__c. Condition: SBQQ__Status__c = "Contracted" AND only when the record changes to meet criteria (prevents re-fires on subsequent edits). Runs after-save.

Element Sequence

6. Get Records: SBQQ__QuoteLine__c where Quote = \$Record AND ProductFamily = "Hardware".
7. Loop the Quote Line collection.
8. Create Asset (in loop): maps Name, AccountId, ContactId (from Quote's Primary Contact), Product2Id, Quantity, Price.
9. On success: Post to Chatter on the Quote ("Assets successfully created").
10. On fault: wrap \$Flow.FaultMessage in a List<String>, call FlowErrorHandler, build a Chatter body with the cleaned message, post the result.

Why a Hybrid (Flow + Apex)?

The orchestration is in Flow because it's visually maintainable by admins extending the solution. The error cleaning is in Apex because regex isn't viable in Flow formulas. Each tool handles what it does best, talking through a thin InvocableMethod interface.

7. Customer Community

A Customer Community on Experience Cloud exposes the backend automation to external users. Four custom Lightning Web Components, one Apex controller.

LWC	Page	Purpose
customerDashboard	Home	KPI tiles: Assets Owned, Active Warranties, Open Service Requests.
customerAssets	My Assets	Read-only datatable; flags assets with active warranties.
customerWarranties	My Warranties	List view + modal registration form with cleaned addError messages.
customerServiceRequests	My Service Requests	Read-only Task list, populated by AssetService.

Security Layers

Account-scoped data isolation is enforced at five layers: License, Profile/Permission Set, OWD, Sharing Set, and defensive Apex (every query filters by the user's AccountId, with WITH SECURITY_ENFORCED on Asset/Warranty queries). For Tasks, which have peculiar Community sharing behavior, a tightly-scoped "without sharing" inner class runs the query using a pre-validated AccountId from the outer "with sharing" controller. The user's identity gates access; the helper executes the query the user is already entitled to make.

8. Environment Notes

Free Developer Org constraints that shaped design choices:

- No external integrations (15K API calls/24h): all logic stays inside Salesforce.
- Limited Customer Community licenses: design assumes Plus license; explicit Permission Set required.
- No Shield Event Monitoring: System.debug used for logging; recommended Application_Log__c covered in Section 9.
- No CI/CD infrastructure: deployment is manual via Salesforce CLI; recommended pipeline covered in Section 9.

9. Future State Recommendations

9.1 Scaling to 1 Million Records

- Asynchronous: move Asset creation to a Queueable chain triggered by Platform Events.
- Batch Apex with scope=200 for one-time backfills.
- Big Objects for warranty history audit (cuts hot-object storage ~80%).
- Custom indexes (via Salesforce Support) on Asset__c and Account__c on the warranty object.
- Skinny tables on Asset hot fields if reports approach timeout.
- Archive Batch: warranties older than 7 years move to Big Object nightly.

9.2 Monitoring & Logging

- Custom Application_Log__c object: structured error capture (Class, Method, Stack, User, RelatedRecord). Replace System.debug calls.
- Tableau CRM dashboard on Application_Log__c: error trends per class, per user, per hour.
- Flow Default Behavior set to email Apex exception address; route to a Slack channel via Outbound Message + middleware.
- Trigger_Setting__mdt custom metadata with an isActive flag per handler — admins can disable a misbehaving handler without a deploy.
- Anonymous Apex heartbeat job: scheduled no-op every 5 minutes; alert on failure.

9.3 CI/CD

- Source control: SFDX project in Git; feature → develop (UAT) → main (prod) branching.
- GitHub Actions on every PR: sfdx force:apex:test:run with --code-coverage; sfdx-scanner static analysis; PMD ruleset for Apex.
- Scratch org per PR: spin up fresh org, deploy feature branch, run all tests — proves clean deploy before merge.
- Promotion: develop deploys to UAT sandbox; main runs validation-only deploy before production deploy (no manual approval clicks).
- Metadata diff bot comments PRs with changed files for explicit review.

10. Deployment Quick Reference

Once metadata is deployed via Salesforce CLI:

```
sf project deploy start -d force-app -o DEVORG
```

```
sf apex test run -o DEVORG --code-coverage --result-format human
```

Manual steps that cannot be in source control are documented in `COMMUNITY_DEPLOYMENT_GUIDE.md`: enable Digital Experiences, create the CPQ Product Rule, create the Permission Set, configure the Sharing Set, set Activity OWD external access.